

<TÍTULO>

LABORATORIO 1: RESTAURAR GRUB DESDE LIVE-CD	2
Requisitos previos	2
Herramientas necesarias	2
Planteamiento	2
Arranque desde CD	3
Identificación de la partición necesaria	3
Reconstrucción del GRUB existente	4
LABORATORIO 2: RESTAURAR GRUB DESDE LA SHELL DE GRUB	6
Requisitos previos	6
Herramientas necesarias	6
Planteamiento	6
Recuperación del GRUB	7
Identificación de mis dispositivos	7
Preparación del entorno de arranque	7
Restablecimiento permanente de GRUB	8

Laboratorio 1: Restaurar GRUB desde LIVE-CD

Requisitos previos

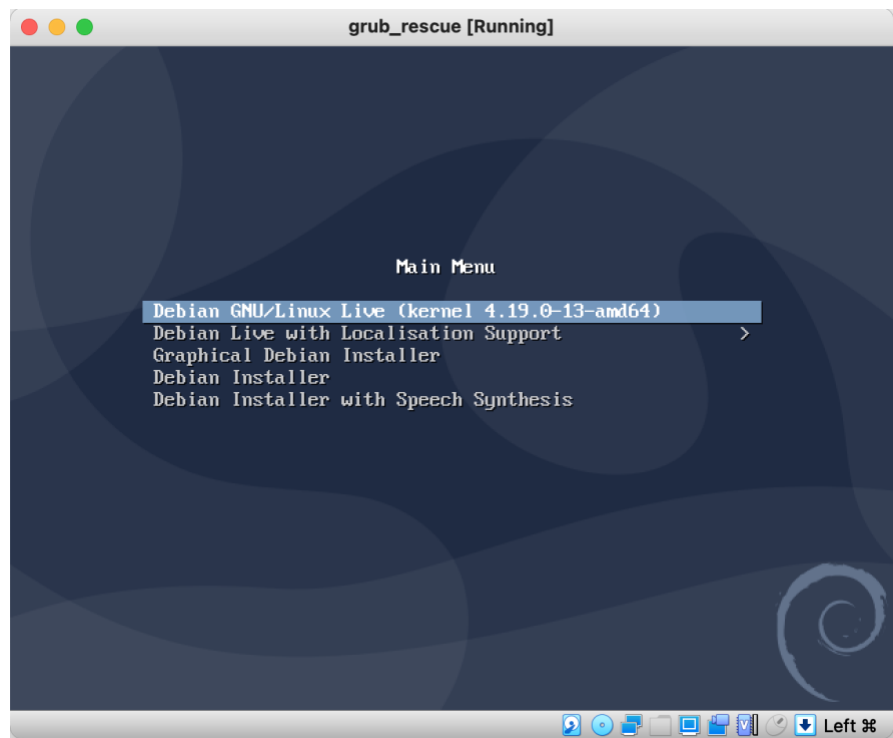
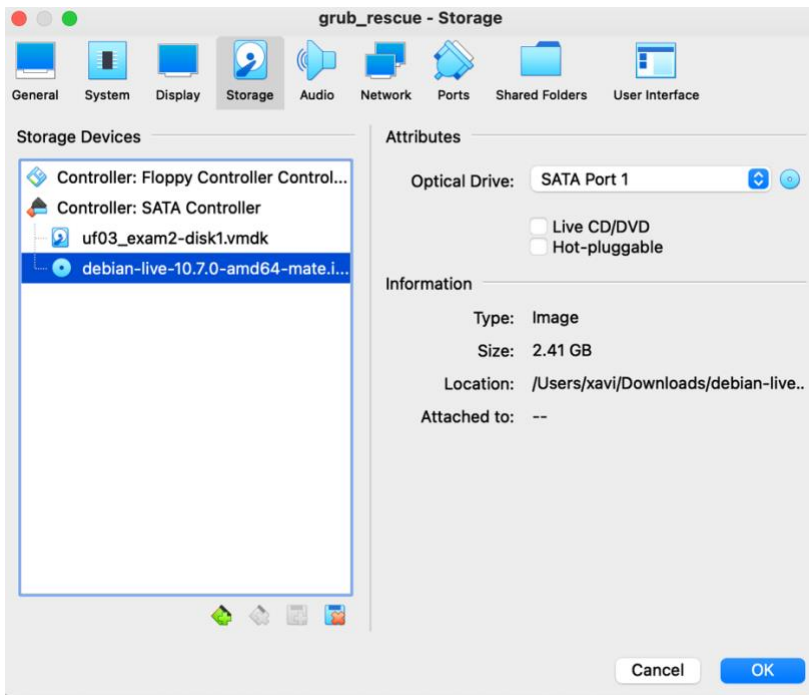
Herramientas necesarias

- ISO Live de Linux

Planteamiento

Tenemos una máquina que no consigue iniciar porque ha perdido la configuración de GRUB que tenía instalada (por motivos que desconocemos).

Necesitamos poder restablecer su configuración, por lo que vamos a acceder desde otro SO (en este caso un LIVE-CD) y vamos a recuperar y colocar en su sitio los archivos que tuviera el sistema anteriormente al motivo del fallo



Arranque desde CD

En primer lugar, insertamos el CD y arrancamos la máquina desde él. Debemos escoger la opción "Iniciar Live" que nos de el menú que aparece.

Identificación de la partición necesaria

Una vez dentro del SO del Live, miramos qué particiones hay en el disco de la máquina, y buscamos qué tiene cada una de ellas (en busca de la partición con los archivos del GRUB)

```
# fdisk -l
Disk /dev/sda: 20 GiB, 21474836480 bytes, 41943040 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x00000000

Device      Boot Start      End  Sectors  Size Id Type
/dev/sda1   *      2048 41943039 41940992   20G 83 Linux

# mount /dev/sda1 /mnt/

# ls /mnt
bin  dev  home  lib32  libx32  media  opt  root  sbin  sys  usr  var
boot  etc  lib   lib64  lost+found  mnt  proc  run  srv  tmp  vagrant

# ls /mnt/boot/
config-4.19.0-16-amd64  initrd.img-4.19.0-16-amd64  vmlinuz-4.19.0-16-amd64
grub                   System.map-4.19.0-16-amd64
```

Vemos que este disco sólo tiene una partición, sda1, y que en ella tenemos el directorio /boot con los archivos de GRUB

Reconstrucción del GRUB existente

Para poder volver a cargar los archivos de esta partición sda1 en el disco sda tenemos que "enjaularnos" dentro de él. Esta técnica nos permite ejecutar comandos del sistema que se encuentra en sda1, y no en el LIVE. La forma de enjaularnos es con el comando "chroot"

```
# chroot /mnt/

# ls /
bin  dev  home  lib32  libx32  media  opt  root  sbin  sys  usr  var
boot  etc  lib  lib64  lost+found  mnt  proc  run  srv  tmp  vagrant
```

Lo que vamos a hacer es instalar el GRUB en el disco sda, pero si trabajamos desde esta nueva "raíz" tenemos un problema. Al no estar operativo, este sistema no ha detectado nada sobre /dev (ni sobre /proc), por lo que estos 2 directorios no contienen la información que necesitamos:

```
# ls /dev/
console  full  ptmx  random  stderr  stdout  urandom
fd  null  pts  shm  stdin  tty  zero

# ls /proc/
```

Para poder trabajar con la información adecuada vamos a necesitar la información que hay en los directorios /dev y /proc del sistema LIVE. Para ello usaremos la opción "--bind" del comando mount que nos permite poder vincular varios puntos de montaje entre ellos:

```
# exit
exit
# mount --bind /dev/ /mnt/dev/

# mount --bind /proc/ /mnt/proc/

# chroot /mnt/

# ls /dev/
autofs          loop-control    shm          tty21  tty41  tty61          vcsa
block           mapper          snapshot    tty22  tty42  tty62          vcsa1
bsg             mem            snd          tty23  tty43  tty63          vcsa2
btrfs-control  memory_bandwidth  sr0          tty24  tty44  tty7           vcsa3
cdrom           mqueue          stderr       tty25  tty45  tty8           vcsa4
char            net             stdin        tty26  tty46  tty9           vcsa5
console         network_latency  stdout       tty27  tty47  ttyS0          vcsa6
core            network_throughput  tty          tty28  tty48  ttyS1          vcsu
cpu_dma_latency null            tty0         tty29  tty49  ttyS2          vcsu1
cuse            port            tty1         tty3   tty5   ttyS3          vcsu2
disk            ppp             tty10        tty30  tty50  uhid           vcsu3
dri             psaux           tty11        tty31  tty51  uinput         vcsu4
fb0             ptmx            tty12        tty32  tty52  urandom        vcsu5
fd              pts             tty13        tty33  tty53  vboxguest      vcsu6
full            random           tty14        tty34  tty54  vboxuser       vfio
```

```

fuse          rfkill          tty15   tty35   tty55   vcs      vga_arbiter
hpet          rtc            tty16   tty36   tty56   vcs1     vhci
hugepages     rtc0           tty17   tty37   tty57   vcs2     vhost-net
initctl       sda              tty18   tty38   tty58   vcs3     vhost-vsock
input         sda1           tty19   tty39   tty59   vcs4     zero
kmsg          sg0            tty2    tty4    tty6    vcs5
log           sg1            tty20   tty40   tty60   vcs6

# ls /proc/
1    158  25   329  6   8      fb    loadavg  stat
10   16   26   33   630  9      filesystems  locks  swaps
100  17   262  330  631  acpi    fs     meminfo  sys
11   175  263  332  634  buddyinfo  interrupts  misc      sysrq-trigger
112  177  267  333  635  bus     iomem   modules  sysvipc
113  178  269  34   64   cgroups  ioports  mounts   thread-self
115  19   27   368  643  cmdline  irq      mtrr      timer_list
117  2    28   369  644  consoles kallsyms  net       tty
12   20   29   373  647  cpuinfo  kcore    pagetypeinfo  uptime
122  21   3    4    648  crypto   keys     partitions  version
13   22   30   40   649  devices  key-users  sched_debug  vmallocinfo
14   225  31   434  675  diskstats  kmsg     schedstat  vmstat
15   23   317  53   678  dma       kpagecgroup  self       zoneinfo
155  237  32   54   691  driver    kpagecount  slabinfo
156  24   320  55   693  execdomains  kpageflags  softirqs

```

Ahora ya disponemos de los datos necesarios, por lo que podemos ejecutar la instrucción que instala el GRUB en el disco sda basándose en los archivos que hemos encontrado:

```

# grub-install /dev/sda --boot-directory /boot
Instalando para plataforma i386-pc.
...
...
Instalación terminada. No se notificó ningún error.

```

Una vez llegados aquí podemos apagar la máquina, quitar el CD y arrancar nuestro equipo para poder iniciar de nuevo desde el menú de GRUB



Laboratorio 2: Restaurar GRUB desde la shell de grub

Requisitos previos

Herramientas necesarias

-

Planteamiento

Tenemos una máquina que no consigue iniciar porque ha perdido la configuración de GRUB que tenía instalada (por motivos desconocidos) pero sí que tenemos acceso a la shell de grub:

```
GNU GRUB version 2.02+dfsg1-20+deb10u4

Minimal BASH-like line editing is supported. For the first word, TAB
lists possible command completions. Anywhere else TAB lists possible
device or file completions.

grub> _
```

Recuperación del GRUB

Lo primero que necesitamos es identificar en qué partición se encuentran los archivos de configuración del GRUB. Para ello debemos listar las diferentes particiones que se han identificado:

Identificación de mis dispositivos

```
grub> ls  
(hd0) (hd0,msdos1)
```

En este caso sólo tenemos una partición, por lo que los ficheros deberían estar en ella

```
grub> ls (hd0,msdos1)/boot/  
config-4.19.0-16-amd64    initrd.img-4.19.0-16-amd64  grub  
vmlinuz-4.19.0-16-amd64  System.map-4.19.0-16-amd64
```

Preparación del entorno de arranque

Definición de variables

Una vez sabemos esto debemos especificar cuál es el prefijo donde se van a localizar los archivos de configuración del GRUB:

```
grub> set prefix=(hd0,msdos1)/boot/grub
```

También definimos la partición a utilizar para el arranque del sistema:

```
grub> set root=(hd0,msdos1)
```

Módulos de kernel necesarios

Una vez tenemos esta configuración cargamos los módulos de kernel necesarios para poder ejecutar nuestro kernel

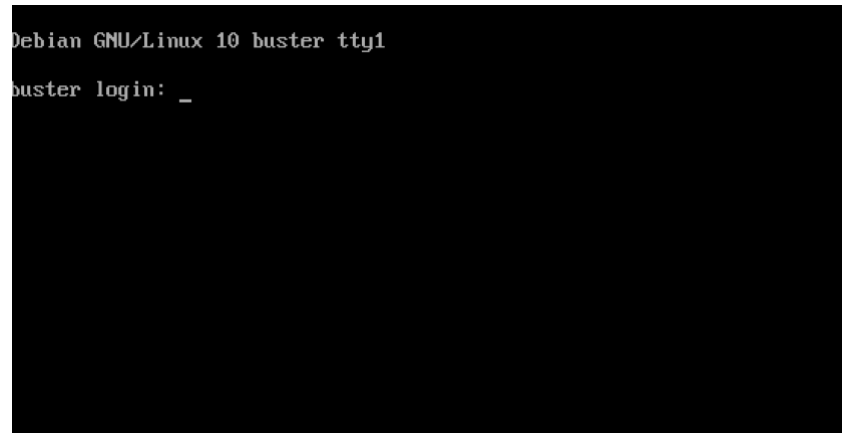
```
grub> insmod normal  
grub> insmod linux
```

Inicio del kernel

Ahora cargamos los ficheros que nos permiten arrancar el kernel y lanzamos el inicio

```
grub> linux /boot/vmlinuz-4.19.0-16-amd64 root=/dev/sda1  
grub> initrd /boot/initrd-4.19.0-16-amd64  
grub> boot
```

De esta manera hemos conseguido volver a tener acceso a nuestro sistema. No obstante, esta configuración es temporal. En el siguiente reinicio volveremos a la shell de grub



```
Debian GNU/Linux 10 buster tty1  
buster login: _
```

Restablecimiento permanente de GRUB

Para poder guardar de nuevo nuestro GRUB en el sistema de arranque de nuestro equipo es necesario volver a instalarlo, a través del comando:

```
# grub-install /dev/sda --boot-directory /boot/
Instalando para plataforma i386-pc.
Instalación terminada. No se notificó ningún error.
```

